

→ initial statement: execute at start of simulation (0 time units) // here, we delay by 10 units per input

→ initial doesn't synthesize to hardware, signals assigned to in initial must be reg

self-checking testbench: more like a unit test, less tedious to verify → check against expected outputs, log failures

```
module testbench2 ();
  reg a, b, c;
  wire y;

  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // apply inputs one at a time
  // checking results
  initial begin
    a = 0; b = 0; c = 0; #10;
    if (y != 1) $display("000 failed.");
    c = 1;
    if (y != 0) $display("001 failed.");
    b = 1; c = 0;
    if (y != 0) $display("010 failed.");
    c = 1;
    if (y != 0) $display("011 failed.");
    a = 1; b = 0; c = 0;
    if (y != 1) $display("100 failed.");
    c = 1;
    if (y != 1) $display("101 failed.");
    b = 1; c = 0;
    if (y != 0) $display("110 failed.");
    c = 1;
    if (y != 0) $display("111 failed.");
  end
endmodule
```

→ \$ is a prefix for system tasks, \$display prints a message to the simulator console

→ only strict equality works well with x and z

using a test vector file is more efficient for more vectors:

000_1
001_0
010_0
011_0
100_1
101_1
110_0
111_0
→ binary text file

→ we use a separate test clock to simulate until file is done

→ \$readmemb reads a binary file into the testvectors array,

```
module testbench3 ();
  reg clk, reset;
  reg a, b, c, yexpected;
  wire y;
  reg[31:0] vectornum, errors;
  reg[3:0] testvectors[10000:0];
  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // generate clock
  always
  begin
    clk = 1; #5; clk = 0; #5;
  end

  // at start of test, load vectors
  // and pulse reset
  initial
  begin
    $readmemb("example.tv", testvectors);
    vectornum = 0; errors = 0;
    reset = 1; #27; reset = 0;
  end

  // apply test vectors on rising edge of clk
  always @(posedge clk)
  begin
    #1; {a, b, c, yexpected} =
      testvectors[vectornum];
  end

  // check results on falling edge of clk
  always @(negedge clk)
  if (~reset) begin // skip during reset
    if (y != yexpected) begin
      $display("Error: inputs = %b", {a, b, c});
      $display(" outputs = %b (%b expected)",
        y, yexpected);
      errors = errors + 1;
    end
    vectornum = vectornum + 1;
    if (testvectors[vectornum] == 4'bxx) begin
      $display("%d tests completed with %d errors",
        vectornum, errors);
    end
  end
  $finish;
end
endmodule
```

\$readmemb does the same for hex files

→ wait one unit of time after rising edge to set new vector and expected output using bit swizzling

→ on falling edge, check if reset not active, error messages if output doesn't match expectation

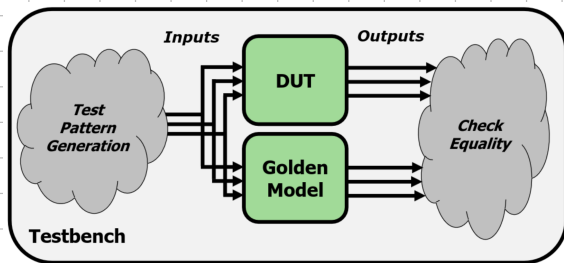
→ \$display has this syntax too: \$display("custom message %0b", {a,b,c})

→ %0b prints the incoming number in binary, %d in decimal

→ \$finish terminates simulation

Automatic Testbenches:

Golden Models: ideal circuit, may be written in other language



→ we have to automatically generate test vectors → sequential coverage or randomization?

pseudocode →

```
module testbench1();
  ... // variable declarations, clock, etc.

  // instantiate device under test
  sillyfunction dut(a, b, c, y_dut);
  golden_model gold(a, b, c, y_gold);

  // instantiate test pattern generator
  test_pattern_generator tgen(a, b, c, clk);

  // check if y_dut is ever not equal to y_gold
  always @(negedge clk)
  begin
    if (y_dut != y_gold)
      $display(...);
  end
endmodule
```

→ this is fully automated, scalable, even timing can be compared with a golden timing model

→ challenges: creating a golden model, test generation, brute forcing tests is inefficient

Van Neumann Model, Processor Design:

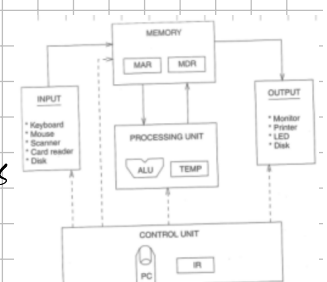
von Neumann Model:

(smallest pieces of work)

→ a program consists of instructions and is contained in memory, the control unit orders instructions

Memory: if a memory has 2^{26} distinct locations and each can store 8 bits

it has an address space of 2^{26} locations and addressability of 8 bits → byte addressable, some ISA's word-addressable



→ reading it: place address in MAR (Memory Access Register) → "interrogate memory"

→ the information stored in that MAR location is then placed in the MDR (Memory Data Register)

→ writing to mem.: put the address in MAR, value in MDR

→ interrogate mem. with $WE=1$ (write enable) → MDR info written to the location we wrote into the MAR

→ MAR locations are permanent, what's inside them isn't

Processing Unit: (PU)

→ the simplest PU is an ALU with ADD, SUBTRACT, basic logic

→ the size of quantities processed in bits is the word length, the element is called a word
(16)

→ a register file stores results temporarily (near the ALU) → LC-3 has 8 16-bit registers for this

I/O devices: peripherals like keyboards and displays

Control Unit: contains Instruction Register (IR) that contains the current ins., then a register that contains the next ins.'s address (program counter, abbreviated to PC)

LC-3 as a Von Neumann machine:

→ filled-in arrowheads → data flows using the signal/bus

→ not filled-in → control signals, no data flows

Memory: our MAR (because of an address space of 2^{16} locations)
and MDR (because of 16-bit words) are 16-bit

I/O: the KBDR (keyboard data reg.) contains the ASCII of struck keys,
the KDSR contains their status

→ the monitor likewise has a Display Data Register and Display Status Reg.

for the same purposes

Processing Unit: consists of ALU and RF

Control Unit: FSM directs activity, processing is done cycle-by-cycle (CLK),

IR is an input for the FSM, the PC saves the next ins.

→ The FSM outputs the 2-bit opcode ALUK, and control signals like GateALU (determines if ALU output pushed onto processor bus)

Instruction Processing:

→ an instruction consists of an opcode (describing what is done) and operands (where it is done)

000	
001	
010	
011	
100	00000110
101	
110	00000100
111	

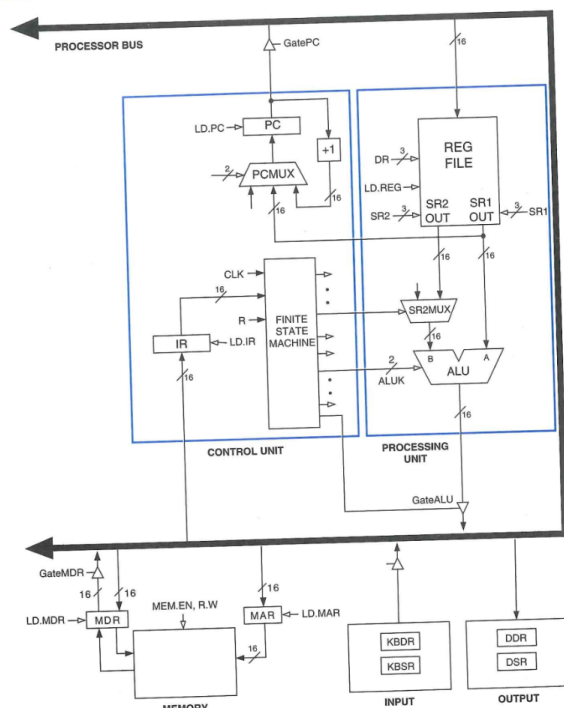
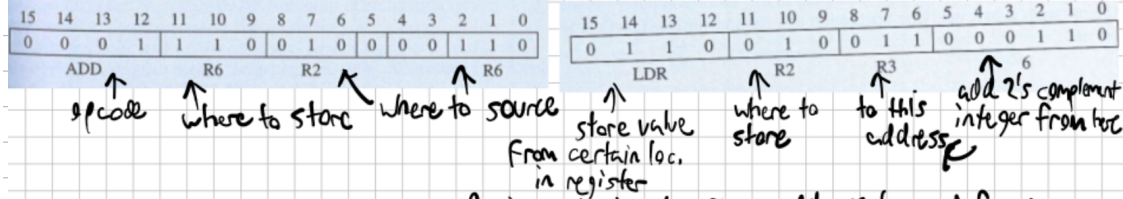


Figure 4.3 The LC-3 as an example of the von Neumann model

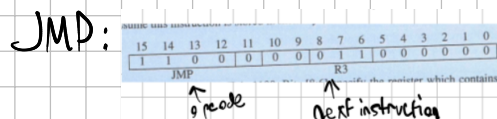


Instruction cycles: how ins. are processed, 6 phases

1. **FETCH**: find next ins., load it into the IR → the program is always in memory → PC contains address of next ins.
 → MAR loaded with PC contents → memory interrogated → next ins. placed into MDR → IR loaded with MDR contents (simultaneously, increment PC)
 → reason: we want PC to contain address of next ins. after the current ins. is processed unless the current ins. doesn't change the control flow
2. **DECODE**: a decoder evaluates the instruction's opcode → 4 to -16 in the LC-3
3. **EVALUATE ADDRESS**: compute address of mem. location needed to process ins.
4. **FETCH OPERANDS**: depends on ins.: for LDR: load MAR with computed mem. address, read mem. → source in MDR for ADD: load source operands from registers
5. **EXECUTE**: result is computed → ADD: ALU is engaged
6. **STORE RESULT**: write result to destination register

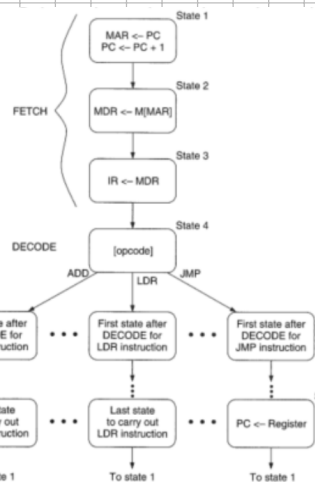
Changing the Execution Sequence:

- ADD is an operate ins., LDR a data movement, control ins. change the sequence of program execution
- Without this, you wouldn't have loops
- each step starts by loading the MAR with the PC → control instructions have to change the PC during a cycle
- they do this in the EXECUTE stage → the next FETCH stage loads the modified PC



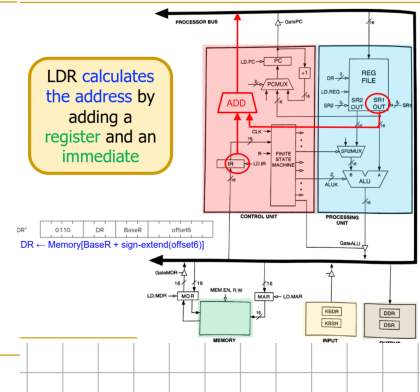
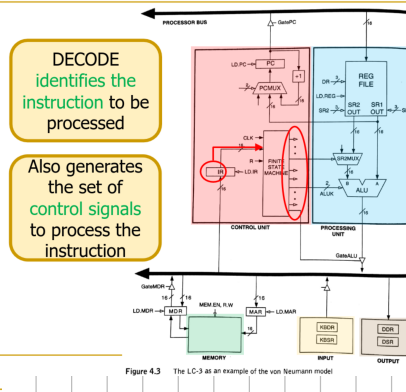
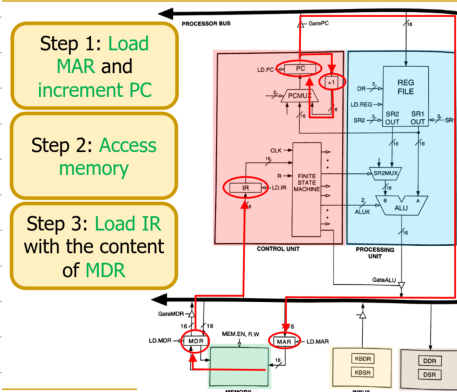
Instruction: "Load PC during EXEC, next ins. processed is from R3"

- important: all steps of any phase are controlled by the FSM
- to load the PC onto the MAR, the FSM has to assert Gate PC and LD. MAR (PC to proc. bus, WE for MAR)
- PC increment: access PCMUX lines to choose +1, assert LD. PC to latch for end of cycle
- State 2: load MDR, state 3: transfer MDR data to IR → assert Gate MDR and LD. IR (latches IR for end of cycle)
- DECODE takes 1 cycle, with the IR, we can take the opcode bits and decide on next state
- Appendix C of P & P has full FSM → enjoy at your own risk
- Stopping a program or the computer is handled by the OS, for the latter the system clock has to stop



Diagrams of instruction cycle:

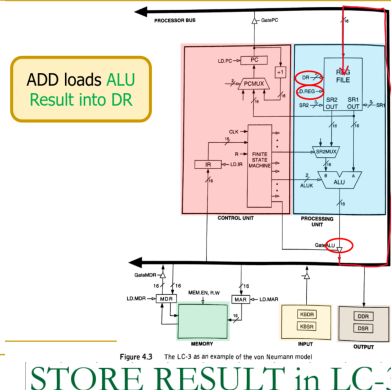
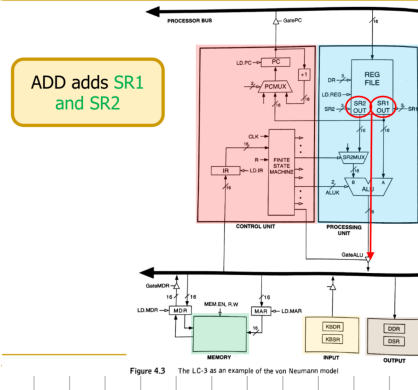
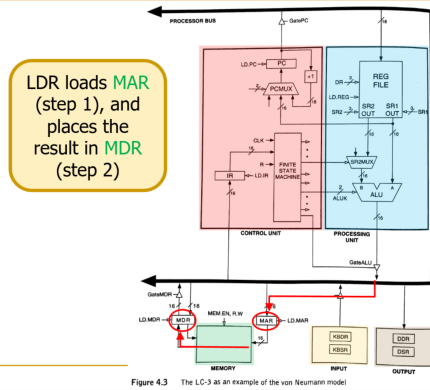
FETCH in an LC-3 Implementation DECODE in an LC-3 Implementation EVALUATE ADDRESS in LC-3



FETCH OPERANDS in LC-3

EXECUTE in an LC-3 Implementation

STORE RESULT in an LC-3 Implementation



STORE RESULT in LC-3

The ISA of the LC-3:

→ an Instruction Set defines opcodes, data types and addressing modes

→ you saw it with ADD, LDR

→ The Instruction Set Architecture (ISA) is the interface between SW and HW

Problem
Algorithm
Program
ISA
Microarchitecture
Circuits
Electrons

- specifies mem. organization
 - address space (LC3: 2^{26} , MIPS: 2^{31})
 - addressability (LC3: 16 bit, MIPS: 32 bit)
 - word- or byte-addressable

→ specifies register set → 8 registers in LC-3, 32 registers in MIPS

→ specifies instruction set (what we discussed)

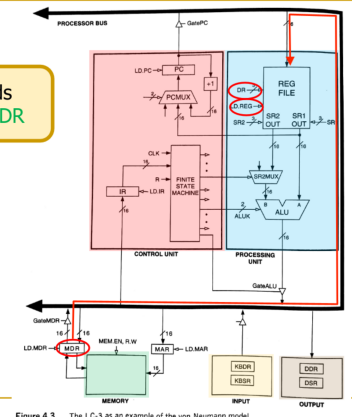


Figure 4.3 The LC-3 as an example of the von Neumann model

→ opcodes: 15 ins. → 4-bit opcodes, one unused → 3 types of ins. as discussed

→ data types: only 2's complement in LC-3, MIPS supports floats and unsigned integers too

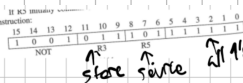
→ addressing modes: ways to locate the operand → 5 modes, will be discussed

→ Condition codes: 3 single-bit reg. (N, Z, P) set/cleared whenever one of the

general-purpose registers is written to depending on if the result is negative, zero, or positive

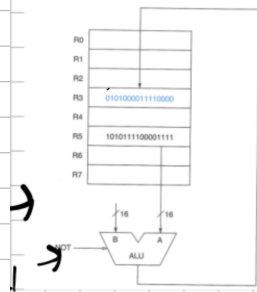
Types of ins.:

Operate ins.: here, ADD, AND, NOT



11 10 9 8 7 6 5 4 3 2 1 0
0 1 1 0 1 0 1 1 1 1 1 1
↑^{dest} store ↑^{src} source
In its, default → NOT is unrx / complements an operand and stores it

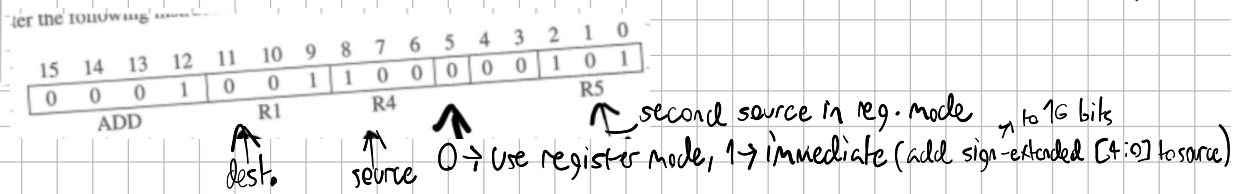
[illegible]



→ B goes unused, control signal to indicate NOT

→ NOT uses the register addressing mode → take operands from gen. purpose regs.

→ ADD and AND are binary → one source and the destination are in register addressing mode

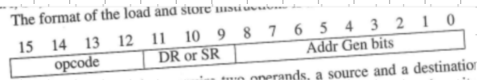


Data Movement:

move info from reg. to mem. → store, from mem. to reg. → load

→ 7 ins. are data movement ins., in any case, the source is unchanged and

destination overwritten → format:



→ 2 operands → one reg., one mem.

→ one is in DR or SR → gen. purpose reg. in [11:0], other in mem.

→ [8:0] address gen. bits → encode info used to compute 2nd operand's address

Addressing Modes:

PC-relative: done by LD and ST

→ bits specify offset relative to PC → after incrementing the PC, sign-extend [6:0] and add it to the PC

→ for LD, this mem. location is loaded into R2 ([11:0])

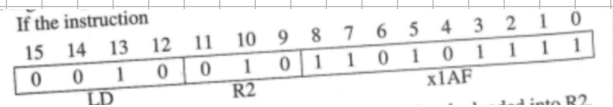
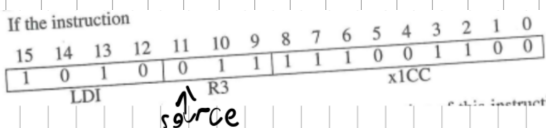
→ add incremented PC to xFAF (sig. ext. of x1AF) here (1)

→ add result to MAR → mem. read → load contents into MDR (2)

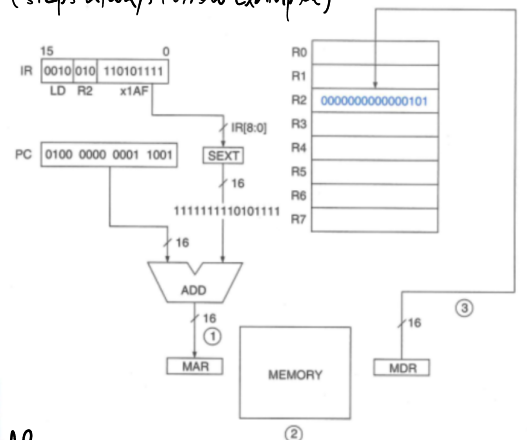
→ then, load it into R2 → mem. operand address limited to 9-bit

Indirect Mode: done by LDI and SDI

→ indirect: the given address in [8:0] contains the operand (in mem.)



(steps always follow example)



→ Steps 1 and 2 identical to PC-relative, step 3: send MDR value to MAR

→ Step 4: reload MDR, step 5: store in gen. purpose reg.

LDI →

Base + offset: LDR, SDR → unlimited range

→ take sig. ext. 6-bit offset, add it to base reg.

→ 1. (from example below): R2 added to sig. ext. offset → load to MAR

2. read mem., load to MDR, 3. load to R1

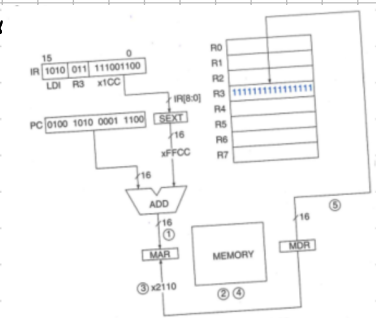
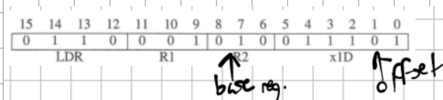


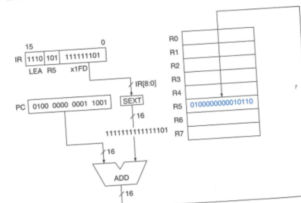
Figure 5.5 Data path relevant to the execution of ADD R1, R4, #2

5.3 Data Movement Instructions

Figure 5.7 Data path relevant to the execution of LDI R3, x1CC

Immediate: only used for LEA

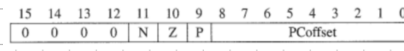
→ load reg. specified by [11:9], load reg. with PC (incremented) + sigext [8:0]



→ doesn't access mem.

Control instructions: break seq. execution

Conditional branch:



→ 11:9 decides branch → examine cond. code regs. where $N/Z/P$ are = 1 in instruction → if any of those is 1, PCoffset is loaded to PC → this happens in the EXECUTE stage

→ FETCH and DECODE are the same as in data movement ins.

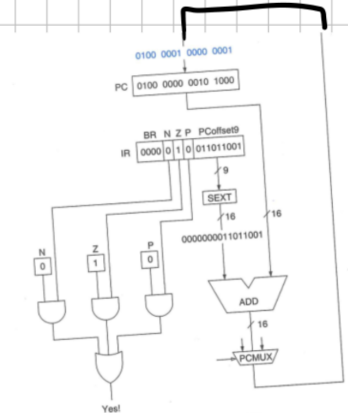
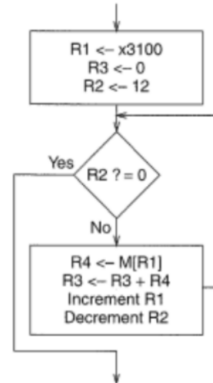
→ repeat loading integer to R3, increment R1, decrement R2 initial variables

→ if R2 = 0 / Z = 1 ⇒ exit loop

loop control:

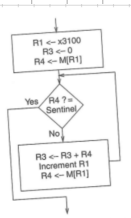
for-loop style: set a counter to n, decrement every round, if 0, continue

while-loop style: choose sentinel (far from input values), if reached, exit loop

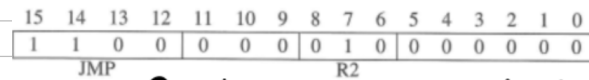


chapter 5 The LC-3

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1	R1 ← x3100
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 ← 0
x3002	0	1	0	1	0	1	0	1	1	0	0	0	0	0	0	0	R2 ← 0
x3003	0	0	0	1	0	1	0	0	1	0	1	1	0	1	0	0	R2 ← 12
x3004	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	BRz x300A
x3005	0	1	1	0	1	0	1	0	0	0	0	1	0	0	0	0	R4 ← M[R1]
x3006	0	0	0	1	0	1	0	1	1	0	0	0	1	0	0	0	R3 ← R3 + R4
x3007	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	R1 ← R1 + 1
x3008	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	R2 ← R2 - 1
x3009	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0	BRnsp x3004



JMP: unconditional branch, limited to 8-bit range here

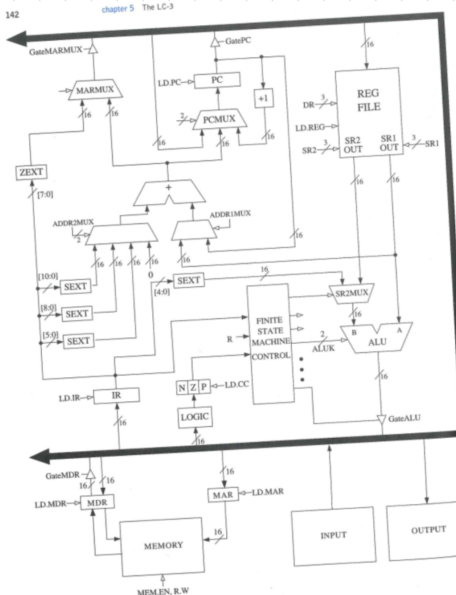


→ load target with [8:0], in execute, that PC is jumped to

TRAP: change PC to a memory address belonging to OS s.t. OS performs a task for the program (Service call)

→ [7:0] contains trapvector (ID for service call)

Information on the LC-3 datapath:



→ black cable → global bus, 16 wires, can carry one value, protected by TSBs

→ Memory: we load the MAR with an address, on load, signals allow mem. read and MDR load, on store, data is put into the MDR and control signals assert WE

→ ALU: source 1: only from RF, source 2: from RF or sig. ext

→ RF: always directly provides source 1, source 2: second reg. vs. sig. ext determined by SR2MUX, which pulls from bit 5 of the ins.

→ PC: supplies next ins. to MAR, supplied by 3-to-1 PCMUX, where incrementation is the rightmost MUX input

→ control ins.: PCMUX source depends on ins. → successful cond. branch: loaded with increment and PCoffset = sig. ext. IR[8:0]

→ adder output is middle PCMUX input for JMP

→ MARMUX: controls source supplying MAR depending on opcode: add either the incremented PC or a base reg. to a literal value or push a zero given by IR

→ ADDR1 Mux gives PC or base reg. / ADDR2 Mux specifies which value is loaded → left input gives zero-extended trapvector

→ I'm leaving out the full FSM of the LC-3 and its exception/interrupt handling as it's excessive for our exam and exception handling will be discussed in a MIPS context

now finally, MIPS assembly!

→ all our LC-3 ins. were in LC-3 assembly, now, we will consider MIPS instructions

→ key differences of MIPS: 32-bit word length, ~~32-bit word length~~, address space of 2^{32} bits, 32 regs. in RF

Code Example 6.1 ADDITION

High-Level Code	MIPS Assembly Code
$a = b + c;$	add a, b, c

→ b, c are source operands, a is the destination operand

→ design principle 1: Simplicity favors regularity

Code Example 6.2 SUBTRACTION

High-Level Code	MIPS Assembly Code
$a = b - c;$	sub a, b, c

→ ins. with consistent number of operands are easier to encode and handle in HW

→ comments are only single-line, made with "#"

→ design principle 2: make the common case fast

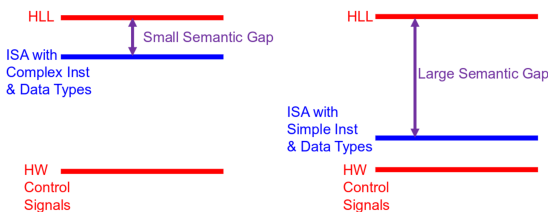
→ MIPS is a RISC → only simple ins., HW used to be decode can be simple and fast, elaborate operations built from simple ins.
↑ reduced ins. set computer

→ ISA's like x86 are CISC, directly supporting complex ins. like inserting into linked lists
↑ complex

→ this is a great place to talk about the semantic gap

Semantic Gap

- How close instructions & data types are to high-level language (HLL)



advantages of complex ins. and data types:

- denser encoding → less code → better memory utilization, saves off-chip bandwidth, better cache hit rate due to better ins. packing (explained later)

- simpler compiler → less need to optimize small ins.

disadvantages:

- larger chunks of work → less optimization opportunities for compiler

- more complex HW → burden of implementing high-level ins. to control sig. falls on HW

Design Principle 3: smaller is faster

→ register files are usually small SRAM arrays with a decoder
source temporary

→ MIPS registers need a \$ prefix, vars. usually in \$0-\$7, \$10-\$17

Name	Number	Use
\$0	0	the constant value 0
\$at	1	assembler temporary
\$a0-\$a1	2-3	procedure return values
\$a0-\$a3	4-7	procedure arguments
\$t0-\$t7	8-15	temporary variables
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	temporary variables
\$k0-\$k1	26-27	operating system (OS) temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	procedure return address

→ MIPS is actually byte-addressable, but we pretend it's word-addressable for now

Memory:

lw and sw:

Assembly Code
This assembly code (unlike MIPS) assumes word-addressable memory lw \$s3, 1(\$0) # read memory word 1 into \$s3
Assembly Code
This assembly code (unlike MIPS) assumes word-addressable memory sw \$s7, 5(\$0) # write \$s7 to memory word 5

Code Example 6.4 REGISTER OPERANDS

High-Level Code	MIPS Assembly Code
$a = b + c;$	# \$s0 = a, \$s1 = b, \$s2 = c add \$s0, \$s1, \$s2 # a = b + c

Code Example 6.5 TEMPORARY REGISTERS

High-Level Code	MIPS Assembly Code
$a = b + c - d;$	# \$s0 = a, \$s1 = b, \$s2 = c, \$s3 = d sub \$t0, \$s2, \$s3 # t = c - d add \$s0, \$s1, \$t0 # a = b + t

→ lw stands for load word, sw for store word, both have the addressing mode base + offset

→ base is a register, offset a parameter → our lw reads from mem. address $\$0 + 1 = 1$ → memory word 1
→ our sw writes contents of \$s7 to mem. address $\$0 + 5 = 5$ → memory word 5

word-addressable mem.

Word Address	Data	
...	...	...
00000003	4 0 F 3 0 7 8 8	Word 3
00000002	0 1 E E 2 8 4 2	Word 2
00000001	F 2 F 1 A C 0 7	Word 1
00000000	A B C D E F 7 8	Word 0

→ real MIPS lw and sw:

→ byte addressable → word address: $4 \cdot \text{word number}$ → if not the case, illegal ins. (not word-aligned)

→ offset can be written in decimal or hex

→ sb and lb ins. can load/store single bytes → introduced later

→ In MIPS, endianness is also arbitrary

→ immediates: addi allows for adding immediates → 16-bit range (there is no subi because you're adding neg. number)

→ Design Principle 4: Good design demands good compromises

→ assembly language are translated to machine language (1's and 0's)

→ variable ins. lengths would add too much complexity, 1 ins. type is too restrictive

→ 3 ins. types as compromise

→ R-type ins.: (Register type, no direct comparison to LC-3)

→ R-type ins. consist of 6 fields → op (opcode), rs (reg. source 1), rt (reg. source 2), rd (reg. dest.), shamt (shift amount, only used for shift ops.), funct (function)

→ R-type ins. have opcode of 0, specific ins. decided by funct field

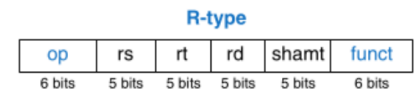
MIPS Assembly Code	
lw \$s0, 0(\$t0)	# read data word 0 (0xABDEF78) into \$s0
lw \$s1, 8(\$t0)	# read data word 2 (0x01EE2842) into \$s1
lw \$s2, 0x(\$t0)	# read data word 3 (0x40F30788) into \$s2
sw \$s3, 4(\$t0)	# write \$s3 to data word 1
sw \$s4, 0x20(\$t0)	# write \$s4 to data word 8
sw \$s5, 400(\$t0)	# write \$s5 to data word 100

High-Level Code

```
a = a + 4;
b = a - 12;
```

MIPS Assembly Code

```
# $s0 = a, $s1 = b
addi $s0, $s0, 4      # a = a + 4
addi $s1, $s0, -12    # b = a - 12
```



Assembly Code

```
add $s0, $s1, $s2
sub $t0, $t3, $t5
```

Field Values

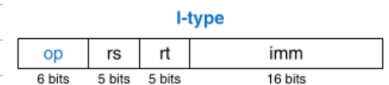
op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
0	11	13	8	0	34

Machine Code

op	rs	rt	rd	shamt	funct
000000	10001	10010	10000	00000	100000
000000	01011	01101	01000	00000	100010

→ I-type ins. (immediate type, just like in LC-3)

→ for fields: op, rs, rt, imm (immediate)



→ op. determined by opcode, rs and imm almost always sources, rt can be source or dest.

Assembly Code

```
addi $s0, $s1, 5
addi $t0, $s3, -12
lw $t2, 32($t0)
sw $s1, 4($t1)
```

Field Values

op	rs	rt	imm
8	17	16	5
8	19	8	-12
35	0	10	32
43	9	17	4

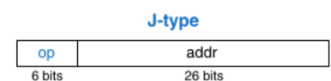
Machine Code

op	rs	rt	imm
001000	10001	10000	0000 0000 0000 0101
001000	10011	01000	1111 1111 1111 0100
100011	00000	01010	0000 0000 0010 0000
101011	01001	10001	0000 0000 0000 0100

→ immediates are sign extended (formally: if imm positive, extended with zeroes, if neg., extend with ones)

→ most MIPS ins. do this, exception: logical ops. (andi, ori, xori) zero extend

J-type: (jump type) → 2 operands, addr (address to jump to), op



→ the idea of storing programs in mem. is called the stored program concept

→ in the fetch stage, the PC is incremented by 4 instead of 1 (byte-addressable mem.)

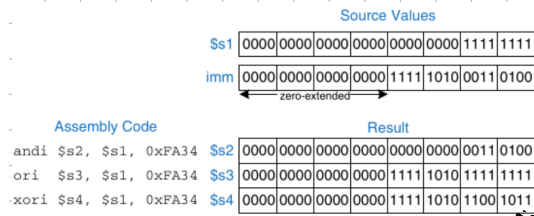
→ in MIPS, the RF and PC hold the architectural state of a system → FSM foreshadowing?

Logical ins.:

→ we have operations like and, or, xor, nor → R-type / binary

→ and can be used for bit masking → and with the extended zero bit forces 0's where you want them

→ logical ops. also exist as I-type → andi, ori, xori



Shift ins.:

→ we can left or right shift by up to 31 bits

→ they multiply/divide by powers of two

→ MIPS has sll (shift left logical), srl, and sra (shift right arithmetic)

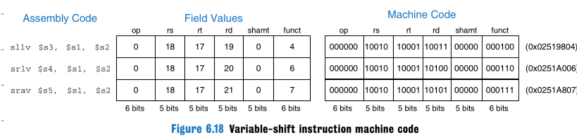
→ left shift fills LSO's with 0's, right shifts can be logical (0's into MSB's) or arith. (sign bit shifts into MSB's)

→ this is the only time shamt is useful

→ left shift → multiply by 2^N , right → divide by 2^N

→ MIPS also has variable-shift ins.

→ sllv (shift left logical variable), srlv, sra



→ rt holds value to be shifted/
Five LSO's of rs give shift amount

→ shamt is all 0's

Generating Constants:

→ addi is good for 16-bit constants, lui and ori for 32-bit

Multiplication/Division:

→ challenge: mult. of 2 32-bit numbers brings 64-bit result

→ special-purpose regs. hi, lo → mult \$0, \$1 places 32 LSO's in lo and MSB's in hi

→ division of 2 32-bit numbers brings 32-bit quotient and 32-bit remainder

→ quotient placed in lo, remainder placed in hi

→ the mflhi reg and mfllo reg ins. move from hi/lo

Branches:

Conditional Branches:

→ MIPS has beq (branch if equal) and bne (branch not equal)

→ assembly uses labels to indicate ins. locations in program

Jumps: (unconditional branches)

→ we have j (jump), jal (jump and link), jr (jump reg.)

→ j is a direct jump, jal also saves a return address, jr jumps to address in register

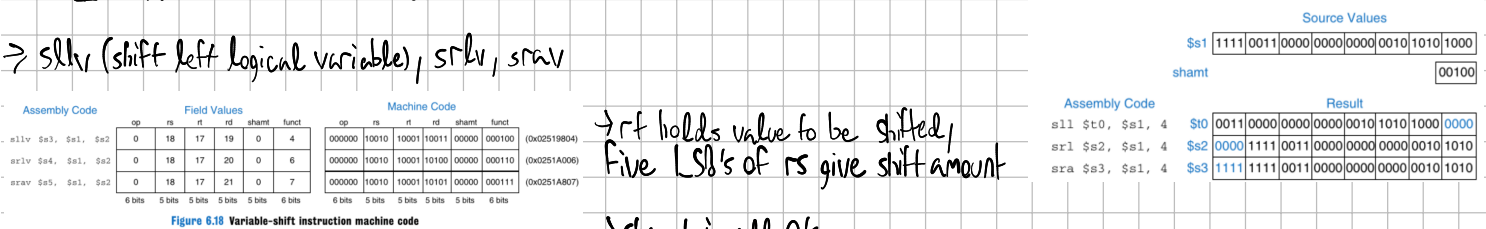
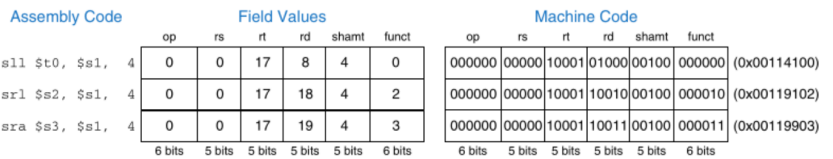
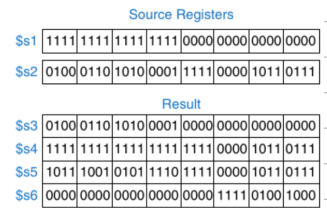
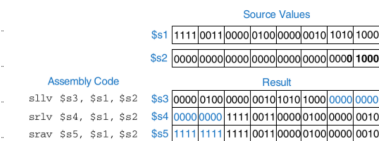


Figure 6.19 Variable-shift operations



Code Example 6.10 16-BIT CONSTANT

High-Level Code	MIPS Assembly Code
int a = 0x4F3c;	# \$s0 = a addi \$s0, \$0, 0x4F3c # a = 0x4F3c

Code Example 6.11 32-BIT CONSTANT

High-Level Code	MIPS Assembly Code
int a = 0x6d5e4f3c;	# \$s0 = a lui \$s0, 0x6d5e # a = 0x6d5e0000 ori \$s0, \$s0, 0x4f3c # a = 0x6d5e4f3c

Code Example 6.12 CONDITIONAL BRANCHING USING beq

MIPS Assembly Code
addi \$s0, \$0, 4 # \$s0 = 0 + 4 = 4
addi \$s1, \$0, 1 # \$s1 = 0 + 1 = 1
sll \$s1, \$s1, 2 # \$s1 = 1 << 2 = 4
beq \$s0, \$s1, target # \$s0 == \$s1, so branch is taken
addi \$s1, \$s1, 1 # not executed
sub \$s1, \$s1, \$s0 # not executed
target: add \$s1, \$s1, \$s0 # \$s1 = 4 + 4 = 8

Code Example 6.13 CONDITIONAL BRANCHING USING bne

MIPS Assembly Code
addi \$s0, \$0, 4 # \$s0 = 0 + 4 = 4
addi \$s1, \$0, 1 # \$s1 = 0 + 1 = 1
sll \$s1, \$s1, 2 # \$s1 = 1 << 2 = 4
bne \$s0, \$s1, target # \$s0 == \$s1, so branch is not taken
addi \$s1, \$s1, 1 # \$s1 = 4 + 1 = 5
sub \$s1, \$s1, \$s0 # \$s1 = 5 - 4 = 1
target: add \$s1, \$s1, \$s0 # \$s1 = 1 + 4 = 5

→ on a jump, all ins. between jump and target are skipped

Conditionals:

→ if statement: skip if condition not met → test opposite condition to high-level code

Code Example 6.14 UNCONDITIONAL BRANCHING USING j

MIPS Assembly Code

```
addi $s0, $0, 4    # $s0 = 4
addi $s1, $0, 1    # $s1 = 1
j    target        # jump to target
addi $s1, $s1, 1    # not executed
sub  $s1, $s1, $s0  # not executed

target:
add  $s1, $s1, $s0  # $s1 = 1 + 4 = 5
```

Code Example 6.15 UNCONDITIONAL BRANCHING USING jr

MIPS Assembly Code

```
0x00002000 addi $s0, $0, 0x2010 # $s0 = 0x2010
0x00002004 jr  $s0              # jump to 0x00002010
0x00002008 addi $s1, $0, 1      # not executed
0x0000200c sra $s1, $s1, 2      # not executed
0x00002010 lw  $s3, 44($s1)     # executed after jr instruction
```

High-Level Code

```
if (i == j)
    f = g + h;

f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
bne $s3, $s4, L1    # if i != j, skip if block
add $s0, $s1, $s2    # if block: f = g + h
L1:
sub $s0, $s0, $s3    # f = f - i
```

→ this pattern continues with if/else:

switch/case: acts like nested if/else

while loops: similar structure to if/else

for loop: also declare a loop variable

High-Level Code

```
int sum = 0;

for (i = 0; i != 10; i = i + 1) {
    sum = sum + i;
}

// equivalent to the following while loop
int sum = 0;
int i = 0;
while (i != 10) {
    sum = sum + i;
    i = i + 1;
}
```

MIPS Assembly Code

```
# $s0 = i, $s1 = sum
add  $s1, $0, $0    # sum = 0
addi $s0, $0, 0      # i = 0
addi $t0, $0, 10     # $t0 = 10

for:
    beq $s0, $t0, done # if i == 10, branch to done
    add $s1, $s1, $s0  # sum = sum + i
    addi $s0, $s0, 1   # increment i
    j    for

done:
```

High-Level Code

```
if (i == j)
    f = g + h;

else
    f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
bne $s3, $s4, else # if i != j, branch to else
add $s0, $s1, $s2  # if block: f = g + h
j    L2            # skip past the else block
else:
    sub $s0, $s0, $s3 # else block: f = f - i
L2:
```

High-Level Code

```
switch (amount) {
    case 20: fee = 2; break;

    case 50: fee = 3; break;

    case 100: fee = 5; break;

    default: fee = 0;
}

// equivalent function using if/else statements
if (amount == 20) fee = 2;
else if (amount == 50) fee = 3;
else if (amount == 100) fee = 5;
else fee = 0;
```

MIPS Assembly Code

```
# $s0 = amount, $s1 = fee

case20:
    addi $t0, $0, 20    # $t0 = 20
    bne $s0, $t0, case50 # i == 20? if not, skip to case50
    addi $s1, $0, 2      # if so, fee = 2
    j    done           # and break out of case

case50:
    addi $t0, $0, 50    # $t0 = 50
    bne $s0, $t0, case100 # i == 50? if not, skip to case100
    addi $s1, $0, 3      # if so, fee = 3
    j    done           # and break out of case

case100:
    addi $t0, $0, 100   # $t0 = 100
    bne $s0, $t0, default # i == 100? if not, skip to default
    addi $s1, $0, 5      # if so, fee = 5
    j    done           # and break out of case

default:
    addi $s1, $0, 0      # charge = 0

done:
```

Code Example 6.19 while LOOP

High-Level Code

```
int pow = 1;
int x = 0;

while (pow != 128) {
    pow = pow * 2;
    x = x + 1;
}
```

MIPS Assembly Code

```
# $s0 = pow, $s1 = x
addi $s0, $0, 1    # pow = 1
addi $s1, $0, 0     # x = 0

addi $t0, $0, 128   # t0 = 128 for comparison
while:
    beq $s0, $t0, done # if pow == 128, exit while
    sll $s0, $s0, 1    # pow = pow * 2
    addi $s1, $s1, 1   # x = x + 1
    j    while
done:
```

magnitude comparison: slt sets rd to 1 if rs < rt, else, rd = 0

Arrays: high-level array is stored in a contiguous memory block

→ accessing an array: load base address into register

Code Example 6.21 ACCESSING ARRAYS

High-Level Code

```
int array[5];

array[0] = array[0] * 8;

array[1] = array[1] * 8;
```

MIPS Assembly Code

```
# $s0 = base address of array
lui  $s0, 0x1000    # $s0 = 0x10000000
ori  $s0, $s0, 0x7000 # $s0 = 0x10007000

lw   $t1, 0($s0)    # $t1 = array[0]
sll  $t1, $t1, 3     # $t1 = $t1 << 3 = $t1 * 8
sw   $t1, 0($s0)    # array[0] = $t1

lw   $t1, 4($s0)    # $t1 = array[1]
sll  $t1, $t1, 3     # $t1 = $t1 << 3 = $t1 * 8
sw   $t1, 4($s0)    # array[1] = $t1
```

→ can be done with lui/ori for 32-bit

→ offset of R-types used to load next el.

→ for loop allows for more efficient access

Address	Data
0x10007010	array[4]
0x1000700C	array[3]
0x10007008	array[2]
0x10007004	array[1]
0x10007000	array[0]

Main Memory

High-Level Code

```
int i;
int array[1000];

for (i=0; i < 1000; i = i + 1) {

    array[i] = array[i] * 8;

}
```

MIPS Assembly Code

```
# $s0 = array base address, $s1 = i
# initialization code
lui  $s0, 0x23B8     # $s0 = 0x23B80000
ori  $s0, $s0, 0xF000 # $s0 = 0x23B8F000
addi $s1, $0, 0      # i = 0
addi $t2, $0, 1000   # $t2 = 1000

loop:
    slt $t0, $s1, $t2 # i < 1000?
    beq $t0, $0, done # if not then done
    sll $t0, $s1, 2    # $t0 = i * 4 (byte offset)
    add $t0, $t0, $s0  # address of array[i]
    lw  $t1, 0($t0)    # $t1 = array[i]
    sll $t1, $t1, 3     # $t1 = array[i] * 8
    sw  $t1, 0($t0)    # array[i] = array[i] * 8
    addi $s1, $s1, 1   # i = i + 1
    j    loop          # repeat
done:
```

→ 1000-element array

→ ith element address computed, stored in \$t0

Address	Data
23B8FF9C	array[999]
23B8FF98	array[998]
⋮	⋮
23B8F004	array[1]
23B8F000	array[0]

Main Memory